

Ros Day02 hw1 AI 사용 보고서

작성자: Doyeon Park

작성일: 2026.09.10

목차

1. AI 보고서 개요
2. AI 사용 부분
3. 깨달은 점

1. AI 보고서 개요

이번 ROS2 과제에서는 커스텀 메시지를 정의하고, Publisher와 Subscriber 사이에서 벡터 데이터를 통신하는 과정에서 ChatGPT를 활용하였다.

처음에는 커스텀 메시지를 어떤 패키지에 만들어야 하는지, CMakeLists.txt와 package.xml에는 어떤 설정이 필요한지 이해하기 어려웠다. 또한 C++의 헤더 파일과 소스 파일을 분리하여 Publisher와 Subscriber를 작성하는 과정도 익숙하지 않았다.

ChatGPT에게 필요한 파일과 터미널 명령을 단계별로 질문하였고, 안내받은 내용을 직접 입력하고 실행 결과를 확인하면서 과제를 진행하였다. 오류가 발생했을 때는 터미널 화면을 전달하고 원인을 질문하여 해결 방법을 찾았다.

2. AI 사용 부분

2.1 커스텀 메시지와 통신 구조 이해

AI에게 과제 내용을 질문하여 메시지 정의용 패키지와 통신 노드용 패키지를 분리하는 방법을 알게 되었다.

parkdoyeon_day2_interfaces 패키지에는 IntVector.msg를 만들고 다음과 같이 메시지를 정의하였다.

```
int32[] data
```

int32는 32비트 정수를 의미하며, 대괄호는 여러 개의 정수를 저장할 수 있는 배열을 의미한다. 배열의 길이가 고정되지 않았기 때문에 입력한 정수의 개수에 따라 서로 다른 길이의 벡터를 전달할 수 있다는 것을 이해하였다.

Publisher는 터미널에서 입력한 정수들을 std::vector에 저장하고, 이를 IntVector 메시지의 data에 넣어 vector_topic으로 발행하도록 작성하였다. Subscriber는 같은 Topic을 구독하고 수신한 벡터를 터미널에 출력하도록 작성하였다.

2.2 오류 원인 확인 및 해결

과제를 진행하면서 패키지가 colcon_ws/src가 아닌 홈 폴더에 생성되는 문제가 발생하였다. AI에게 터미널 화면을 보여주고 질문한 결과, 패키지 생성 명령을 실행한 위치가 잘못된 것을 확인하였다. 생성된 패키지를 colcon_ws/src로 이동하여 해결하였다.

C++ 패키지 생성 과정에서는 package_name이 필요하다는 오류가 발생하였다. --dependencies 뒤에 패키지 이름을 입력하여 새 패키지 이름까지 의존성으로 인식된 것이 원인이었다. AI의 안내에 따라 패키지 이름을 명령 앞부분으로 옮긴 뒤 정상적으로 생성할 수 있었다.

빌드가 성공한 후에는 ros2 run 명령에서 “No executable found”가 출력되었다. 처음에는 빌드 문제라고 생각했지만, AI의 안내에 따라 CMakeLists.txt와 파일 저장 여부를 확인하였다. 확인 결과 수정한 CMakeLists.txt가 저장되지 않은 상태였다. 파일을 저장하고 다시 빌드한 뒤 vector_publisher와 vector_subscriber가 정상적으로 실행되었다.

잠깐, 패키지는 잘 만들어졌는데 위치가 **홈 폴더로 잘못 잡혔어**. 화면에 다음 문구가 보여.

채팅에 추가

destination directory: /home/doyeon

현재 패키지는 `~/parkdoyeon_day2_interfaces` 에 있고, `~/colcon_ws/src` 에는 아직 없어. 다시 만들 필요 없이 옮기면 돼.

터미널에서 실행해줘.

```
</> Bash
mv ~/parkdoyeon_day2_interfaces ~/colcon_ws/src/
cd ~/colcon_ws/src
pwd
ls
```

마지막 결과에 `parkdoyeon_day2_interfaces` 가 보이면 정상 위치로 이동된 거야. 결과를 보여주면 다음 단계에서 `msg` 폴더를 만들자.

(수요일) 오후 8:22

내가 강 옮김

좋아. 이제 커스텀 메시지 파일을 만들자.

터미널에서 실행해줘.

```
</> Bash
cd ~/colcon_ws/src/parkdoyeon_day2_interfaces
mkdir -p msg
touch msg/IntVector.msg
code .
```

VS Code가 열리면 왼쪽에서 `msg` → `IntVector.msg` 를 열고 아래 한 줄을 입력해서 저장해.

채팅에 추가

int32[] data

- `int32` : 정수 자료형
- `[]` : 여러 정수를 담는 가변 길이 배열
- `data` : 벡터를 담을 변수 이름

저장한 다음 터미널에서 확인해줘.

```
</> Bash
cat msg/IntVector.msg
```

결과에 `int32[] data` 가 나오면 다음으로 `CMakeLists.txt` 를 수정하자.



지금 CMakeLists.txt 는 정확해. 저장됐으면 다음은 package.xml 이야.

왼쪽에서 package.xml 을 열고 이 줄을 찾아.

```
</> XML
<buildtool_depend>ament_cmake</buildtool_depend>
```

바로 아래에 두 줄을 추가해.

```
</> XML
<buildtool_depend>roscpp</buildtool_depend>
<exec_depend>roscpp</exec_depend>
```

그리고 아래쪽의 <export> 가 시작되기 바로 위에 이 줄을 추가해.

```
</> XML
<member_of_group>roscpp</member_of_group>
```

관련 부분이 다음처럼 되면 돼.

```
</> XML
<buildtool_depend>ament_cmake</buildtool_depend>
<buildtool_depend>roscpp</buildtool_depend>
<exec_depend>roscpp</exec_depend>

<member_of_group>roscpp</member_of_group>

<export>
  <build_type>ament_cmake</build_type>
</export>
```

저장하고 package.xml 화면을 보내줘. 다음 단계에서 메시지 패키지만 먼저 빌드해볼게.

📄 🗑️ ↩️ (수요일) 오후 8:28

3. 깨달은 점

이번 과제를 통해 ROS2 에서는 필요한 메시지 형식을 직접 정의하여 사용할 수 있다는 것을 알게 되었다. 또한 Publisher 와 Subscriber 가 같은 Topic 이름과 메시지 형식을 사용해야 정상적으로 통신한다는 것을 이해하게 된 것 같다.

AI 를 사용하면서 어떤 내용을 어느 파일에 작성해야 하는지 단계별로 확인할 수 있었다. 또한 AI 의 도움을 받아 오류가 발생했을 때 오류 메시지와 현재 화면을 함께 확인하며 원인을 더 쉽게 찾을 수 있었던 것 같다.

이번 과제를 통해 커스텀 메시지 작성 방법과 C++ Publisher 및 Subscriber 의 기본 구조를 이해할 수 있었던 것 같다.